

Laser Vault Heist

Computer Graphics Fundamentals

Final Project Report

Student: Nurbek Yerbulekov

Group: SE-2428

Topic 12
Standalone C++17 and OpenGL 3.3 core game

Table of Contents

1 Introduction 3

2 Literature Review and Background 4

3 Project Description 9

4 Technical Implementation 10

4.3 Algorithms and system architecture 11

5 Graphics and Rendering 14

5.2 Lighting and shading 15

5.3 Texturing and animation appearance 17

6 Testing and Evaluation 18

6.2 Performance evaluation 19

6.3 Issues and debugging 20

7 Conclusion and Future Work 21

8 References 22

9 Appendix 26

1 Introduction

Laser Vault Heist is a complete first-person stealth exercise built around one compact mansion vault. The player reads the motion of six laser barriers, moves through the room, crouches below a final low beam and opens a mechanical safe. A laser contact changes the entire lighting state to red and offers a fresh attempt. This report explains the implementation, compares its methods with graphics literature and records reproducible local evidence.



Figure 1. The original vault in the actual OpenGL runtime

Figure 1 shows the visual target in the running application: warm wood and plaster, cold window light, restrained brass and a readable route. Geometry, UV textures, character rig and animation were authored in Blender. The renderer, lighting, scene hierarchy, collision and playback are custom C++/GLSL systems; no game engine performs these tasks.

The outcome is a reproducible source project and a standalone Windows build with all three topic stretch goals: rotating spotlights, live CCTV and safe-opening victory. Evidence includes tests, a continuous captioned demo and three performance runs. A local result cannot establish performance on an unknown TA computer or replace a live demonstration of understanding.

2 Literature Review and Background

2.1 Selection and journal evidence

Eighteen journal-indexed articles provide the background below. Titles, authors, venues and DOIs were checked against publisher-deposited Crossref metadata and publisher or author pages. Sixteen remain even if the two Communications of the ACM articles are excluded. Methods are compared with actual implementation choices; citing an alternative does not mean that it was implemented.

Journal	SJR 2025	Category
ACM Transactions on Graphics	Q1	Computer Graphics and Computer-Aided Design
Computer Graphics Forum	Q1	Computer Graphics and Computer-Aided Design
IEEE Transactions on Visualization and Computer Graphics	Q1	Computer Graphics and Computer-Aided Design
Computers & Graphics	Q2	Computer Graphics and Computer-Aided Design
Communications of the ACM	Q1	Computer Science (miscellaneous)

The ranking year is 2025, not the historical publication year of every article. Quartiles were cross-checked through named public SJR reproductions listed with the references. Direct SCImago and Scopus access returned HTTP 403, so direct access is not claimed. Article identity is distinct from journal ranking provenance. Machine-readable records and both ranking links per journal are retained in references-verified.json and source-audit.json.

2.2 Lighting and material models

Phong [1]. Phong compares shading rules and relates image quality to normals, hidden-surface removal and perception. The project evaluates lighting per fragment. It uses the Blinn half-vector variant rather than copying Phong's original reflection-vector expression.

Cook and Torrance [2]. Cook and Torrance model reflection using surface microgeometry and material-dependent directional response. This is the physical comparison for the vault's brass and painted metal. The implemented empirical Blinn-Phong shader is cheaper and easier to explain, but it does not reproduce a calibrated metal BRDF.

Schlick [3]. Schlick develops a compact reflectance model that balances physical constraints with practical cost. Its efficiency goal supports keeping material controls understandable. The game does not implement the full Schlick BRDF or claim physically based rendering.

These papers separate a shading evaluation strategy, a physical reflectance model and an efficient approximation. Laser Vault Heist needs legible materials and a shader that can be explained line by line. It evaluates empirical Blinn-Phong lighting per fragment, with explicit diffuse color, specular strength and shininess. Brass is recognizable through shape, palette and highlights, but is not a measured conductor material.

This choice also permits artistic control. Warm lamps and a cold moon source can be tuned without hiding the required dynamic illumination in baked textures. Physically calibrated reflectance would need new implementation and validation; the current shader is not described as PBR.

2.3 Skeletal deformation and animation

Kavan et al. [4]. Kavan and colleagues replace linear matrix blending with approximate dual-quaternion blending to reduce collapsing-joint artifacts. The thief uses linear blend skinning because its weights and motions are simple. This paper identifies a concrete future improvement for shoulder and knee volume preservation.

Kavan et al. [5]. Kavan, Sloan and O'Sullivan fit standard linear skinning to animated meshes with sparse weights and efficient optimization. The runtime likewise benefits from sparse influences and a compact palette. Our weights are authored procedurally and normalized, not fitted with the paper's optimization.

Le and Deng [6]. Le and Deng jointly recover rigid bone transforms and sparse convex skinning weights from example poses. Their nonnegative, normalized weight constraints motivate explicit export validation. The project keeps an authored anatomical hierarchy instead of performing skinning decomposition.

Merry et al. [16]. Merry, Marais and Gain generalize skeletal subspace deformation while retaining a linear formulation and hardware rendering. The comparison explains why plain linear blend skinning can lose volume around strongly bent joints. Our constrained animation range and garment shapes mitigate, but do not mathematically eliminate, those artifacts.

Mohr and Gleicher [17]. Mohr and Gleicher fit a compact real-time deformation model to examples exported from richer authoring tools. The project similarly separates Blender authoring from a small runtime representation. It exports authored transforms directly, rather than fitting examples or reproducing muscle deformation.

James and Twigg [18]. James and Twigg derive skinning approximations from deformable mesh sequences using automatically chosen transforms and weights. Their work supports the practicality of GPU matrix palettes. Our 20-bone rig is explicitly authored and hierarchical, so no automatic bone discovery or progressive approximation is needed.

Together these works explain the selected compromise: a small authored hierarchy and sparse normalized weights give inexpensive GPU deformation. Fitting or dual-quaternion methods address quality limits that simple matrix blending does not remove. The project validates its limited authored poses rather than claiming a general deformation solution.

2.4 Collision and visibility

Klosowski et al. [7]. Klosowski and colleagues organize discrete-orientation bounding volumes into hierarchies for efficient collision queries in complex environments. A hierarchy is unnecessary for six laser boxes and a small set of furniture colliders. Transforming body samples into each beam's local space keeps the project inexpensive and directly inspectable.

Jiménez et al. [8]. Jimenez, Thomas and Torras classify continuous and sampled collision strategies and explain how temporal and spatial pruning control cost. The game adopts repeated static local-box tests at 240 Hz, with a conservative body margin. This is bounded sampling, not an exact continuous-collision solver for arbitrary velocities.

Teschner et al. [13]. Teschner and colleagues survey deformable collision using hierarchies, distance fields, spatial partitioning, image-space and stochastic methods. The animated thief is represented by a sampled upright body for gameplay. This avoids deforming triangle-level collision, while pose-bound checks keep the crouched mesh compatible with the chosen clearance.

Cohen-Or et al. [9]. Cohen-Or and colleagues survey visibility algorithms for interactive walkthroughs, including techniques that avoid drawing hidden geometry. The compact single room uses depth testing, back-face culling and static instancing. More complex occlusion systems would add overhead without a demonstrated benefit at this scene size.

Visual complexity is separated from gameplay queries. Hundreds of decorative objects do not require hundreds of triangle tests. Large furniture uses simple spatial proxies, and each of six beam transforms supports a few upright body samples. A fixed 240 Hz simulation bounds ordinary motion between tests. This remains sampled detection: extreme motion or an unbounded timestep would require a continuous method.

Static instancing addresses repeated visual work without an occlusion hierarchy. Measurements test whether that choice is sufficient on the available computer. They do not establish the same result for a larger room or the unknown TA machine.

2.5 Display, shadows and texture sampling

Reinhard et al. [10]. Reinhard and colleagues adapt photographic tone reproduction to map high-dynamic-range images to a display. The project also renders into a floating-point target before display conversion. Its implemented global fitted curve is credited separately and is not Reinhard's photographic operator.

Larson et al. [11]. Larson, Rushmeier and Piatko preserve visibility through tone reproduction informed by luminance adaptation and human vision. This provides a criterion for tuning the night scene: the safe and hazards must remain readable. The game uses artistic tuning rather than the paper's histogram and visual-adaptation model.

Hasenfratz et al. [12]. Hasenfratz and colleagues compare real-time shadow methods by geometry, lighting conditions, image quality and cost. The vault uses one directional depth map with a small PCF kernel. The spotlights illuminate surfaces but do not each render a separate shadow map; their dust cones are an inexpensive visual approximation.

Blinn and Newell [14]. Blinn and Newell use surface parameterization to map patterns and filter their contributions over a pixel footprint. The project's UV coordinates and mipmaps address the same sampling problem. Its mesh rasterizer does not reproduce the original patch renderer or environment-reflection system.

Lefebvre and Hoppe [15]. Lefebvre and Hoppe synthesize controllable, deterministic textures through parallel neighborhood matching and multiresolution operations. The game instead generates seeded painted textures offline. This produces reproducible wood, cloth and plaster without the runtime complexity of exemplar-based synthesis.

The design criterion is visible information at bounded cost. Glow makes thin laser geometry readable, one shadow map grounds the night scene, and mipmaps reduce aliasing. Global tone mapping does not model human adaptation, additive cones are not volumetric transport, and painted textures do not implement exemplar synthesis. These limits are retained in the evaluation.

3 Project Description

3.1 Objective

The objective is to implement topic 12 as a playable, explainable computer-graphics project with its complete Core and all three Stretch goals. The player must fail, retry and win through the same rules used by the demonstration. Each feature should trace to inspectable assets, source code and a test or capture.

3.2 Problem statement

A thin rotating laser exposes several correctness problems. Its rendered orientation and contact volume must agree; a crouching camera must match the body proxy and animated mesh; a second camera must see the character without recursively sampling its own target. Movement must remain stable across frame rates, and the safe hierarchy must produce a coherent mechanical opening.

3.3 Scope

The room is approximately 8 by 12 by 4 metres. It contains wood flooring and beams, plaster and stone surfaces, a night window, deposit drawers, desk and chair, lamp, monitor, framed art, rug, plants and a grey safe with brass details. Six laser barriers occupy the route. The thief wears a brown jacket over a hooded zip sweatshirt, brown denim and low shoes, with visible skin and five-finger hands.

The scope is one room and one player. Guards, combat, weapons, multiplayer, campaigns and shops are excluded. Sound was optional and is not implemented. This is the brief's single-author teaching example; the older PDF's team size does not justify invented participants or classroom history.

Interaction	Effect
Desk lamp	Toggles local illumination and visible glow
Security monitor	Switches between two real CCTV cameras
Safe	Turns wheel, retracts bolts, swings door and wins

4 Technical Implementation

4.1 Tools and technologies used

Component	Role / verified version
C++17 / GLSL 330	Game rules and own OpenGL shaders
GLFW 3.4 / GLAD 2.0.8	Window, input, context and generated GL loading
GLM 1.0.1	Vector, matrix and quaternion operations
Dear ImGui 1.91.9b / stb	UI, image decoding and screenshots
Blender 5.2.3 LTS	Geometry, UVs, rig, actions and export
Clang 23.1.4 / CMake 3.31.6	LLVM-MinGW 20261020, Ninja 1.11.1
Python / FFmpeg	Asset/evidence scripts and optional recording

4.2 Development process

Development progressed from an executable greybox and collision tests to Blender assets, skinning, rendering refinements and measured evidence. Actual Git commits preserve those changes; historical milestones and team contributions were not manufactured. Modeling/export scripts were executed through Blender MCP and retained. The final application runs without Blender or MCP.

The course starter was not supplied. C++17 and CMake follow the brief's permitted baseline. All C/C++ dependencies are vendored; no download occurs during configuration, build or first run. Windows LLVM-MinGW was verified. MSVC and other operating systems have instructions but no claimed execution result.

4.3 Algorithms and system architecture

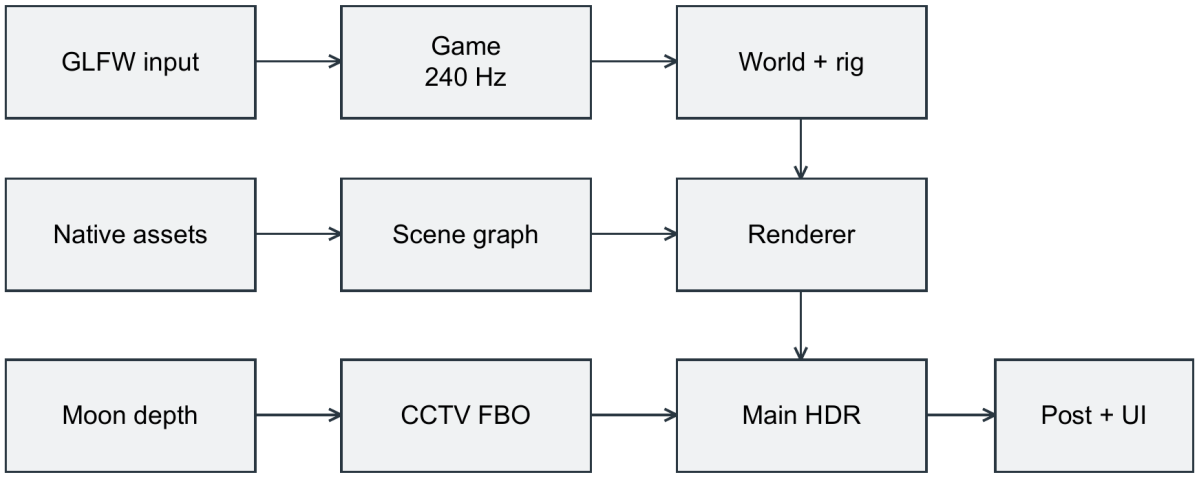


Figure 2. Data flow from input and exported assets to the displayed frame

Figure 2 separates simulation from rendering. main.cpp collects input and manages the window. Game advances rules in fixed steps; world.cpp applies results to nodes and animation selection. Scene loads manifests and builds static instance groups. Shader, Mesh and RenderTarget own GL resources. Renderer schedules passes, Animator evaluates the skin palette, and UI reads the actual state.

Interaction and retry edges are retained until a simulation step consumes them. Above 240 render FPS, a short edge can otherwise fall between updates. Pause stops simulation and animation. A minimized framebuffer waits for events instead of rendering to a zero-sized target.

Menu, Playing, Alarm, Victory and Pause have explicit transitions. Retry resets position, laser phases, lamp/CCTV settings and safe timing. Director is a deterministic QA input producer; it cannot teleport the player or disable normal collision rules. The final demo is a reproducible live run through those rules.

4.3.1 Transformations and laser contact

With column vectors, world = parentWorld * T * R * S and clip = P * V * world * p. The rightmost operation acts first. Normals use transpose(inverse(mat3(world))) and normalize again per fragment, which is necessary under non-uniform scale. Parents precede children in the scene graph.

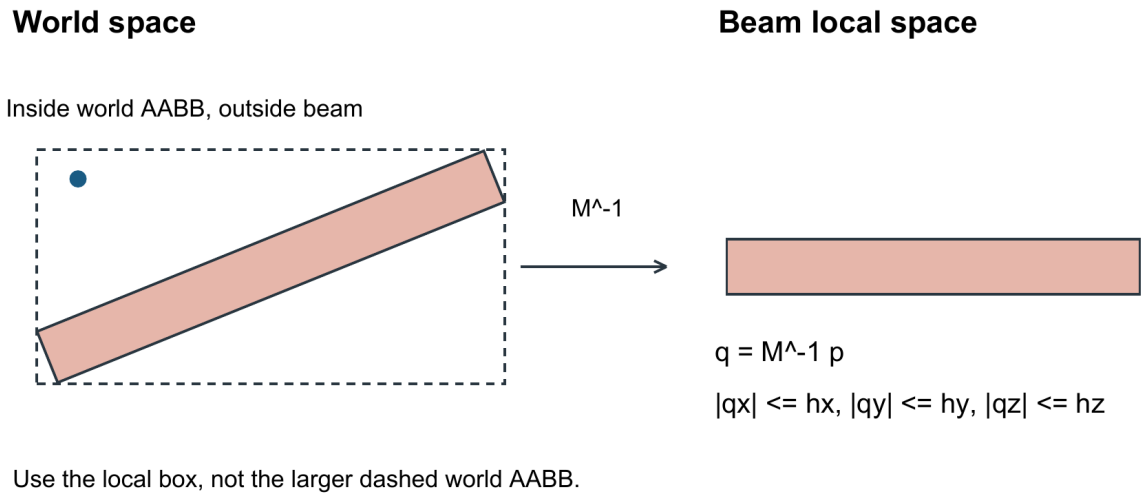


Figure 3. An inverse transform reduces the oriented beam query to a local box test

Figure 3 shows the world-AABB false-positive problem. For beam M and body sample p, q = inverse(M) * vec4(p,1). Contact requires |qx| <= hx, |qy| <= hy and |qz| <= hz. The implementation adds a 0.16, 0.055, 0.16 metre local margin and samples the body every 0.08 m from 0.12 m, plus its exact top. Rendering and collision share the beam matrix.

```
glm::vec3 p = glm::vec3(glm::inverse(model) * glm::vec4(point, 1));
return glm::all(glm::lessThanEqual(glm::abs(p),
    halfSize + glm::vec3(1e-6f)));
```

A left-wall pivot supplies the rotation centre. Five barriers move or rotate; the final 1.42 m beam requires crouching. Fixed 1/240 s steps reduce ordinary tunneling, but the sampled body is not an exact continuous collision solver.

4.3.2 Animation and coordinate conversion



Figure 4. Runtime T-pose and corrected crouch-walk pose in neutral lighting

Figure 4 checks the same exported mesh in rest and crouched poses. It has 21,782 triangles, 20 bones and five actions: Idle, Walk, CrouchIdle, CrouchWalk and Interact. Weights sum to one; the format supports four influences and the assets use at most two. Local translation/scale lerp and quaternion slerp interpolate samples; transitions blend over 0.2 s.

Skinning applies p' = sum_i(w_i * G_i(t) * B_i^-1 * p), with current global bone G and inverse bind B^-1. The controller supplies world movement; clips remain in-place. Walk phase follows travelled distance using 1.2 m and 0.75 m strides. A crouched interaction overlays arm motion without raising the root.

Blender (x,y,z) maps once to runtime (x,z,-y); transforms and inverse binds use C*M*C^-1. Native LVHM/LVHA formats document each field, while verified GLBs remain exchange assets. Body heights 1.90/1.29 m cover sampled mesh maxima 1.8909/1.2755 m.

5 Graphics and Rendering

5.1 Rendering techniques and live CCTV



Figure 5. The physical monitor and enlarged view display a live second camera

Figure 5 is a frame from the real-time demo. A 640 by 360 framebuffer has color and depth attachments; completeness is checked. Its camera renders the room and complete animated thief, omitting the monitor screen to avoid texture feedback. E changes the monitor camera. The main first-person pass hides the full body to prevent near-camera clipping.

The renderer uses indexed VAO/VBO/EBO drawing and static instancing. Mesh/material groups receive their transforms once; dynamic safe parts, fixtures and bones update through uniforms. Textures and static mesh buffers are not reloaded each frame. Destructors release resources before the GL context is destroyed.

Passes are a 2048-square moon depth map, CCTV, the main RGBA16F target, post-processing and ImGui. A separate display conversion makes enlarged CCTV suitable for ImGui. The physical monitor samples linear color within the main pipeline, avoiding a second gamma conversion.

5.2 Lighting and shading



Figure 6. Rotating mounts, visible cones and surface spot lighting

Figure 6 shows two spotlights whose positions and directions follow their mount matrices. Six other local lights and a directional moon illuminate the room. Local attenuation is power/(1 + 0.20d + 0.095d^2); smoothstep between cone cosines softens spotlight edges.

For normal N, light direction L and view direction V, H = normalize(L + V). Per-fragment lighting uses base*max(N.L,0) + ks*max(N.H,0)^s, multiplied by color, attenuation and cone strength. Specular is suppressed on the unlit side. Ambient and emission are separate. The moon uses a biased 3 by 3 PCF depth comparison; local lights are unshadowed.

This is empirical Blinn-Phong, not PBR [1]-[3]. Additive depth-tested cone meshes visualize dust; they do not integrate volumetric scattering. The HDR image receives modest glow and Narkowicz's credited fitted curve [19], followed by gamma conversion and vignette. It is not the photographic operator discussed in [10].

5.2.1 Alarm and safe hierarchy

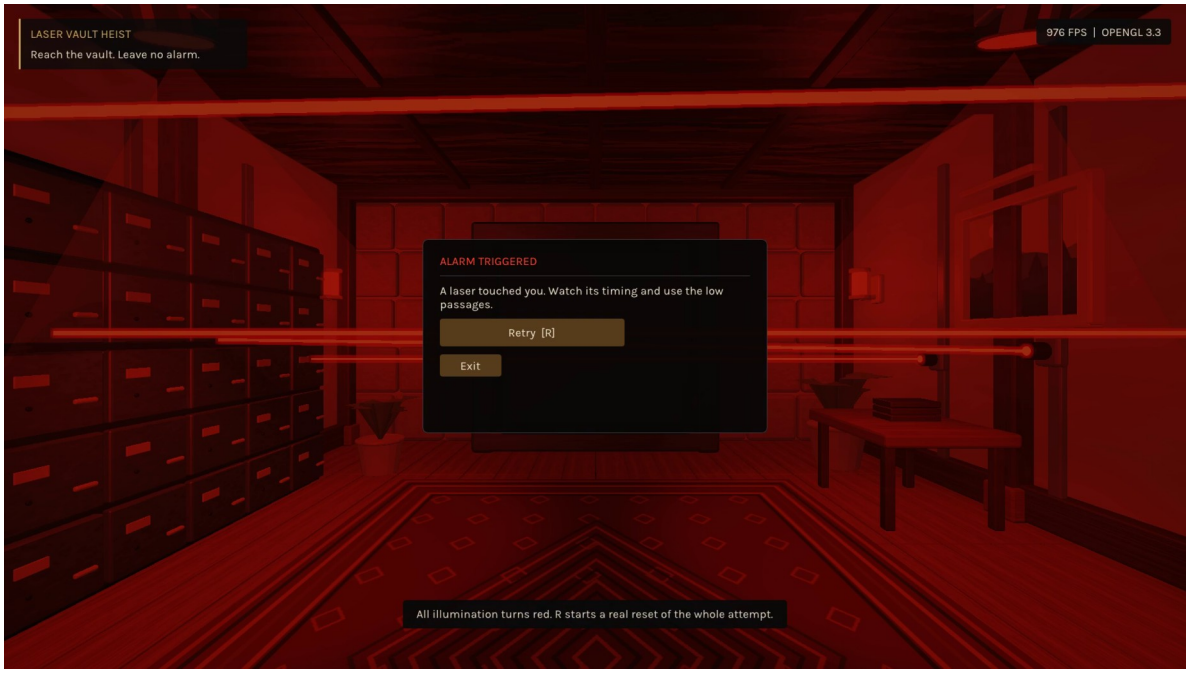


Figure 7. Real contact changes illumination and feedback to red



Figure 8. The final attempt ends with the safe open and a real Victory state

Figures 7 and 8 show failure and success in the same executable. Alarm recolors local lights, moon/ambient and emissive fixtures. Opening turns the wheel 150 degrees, retracts bolts 0.22 m and rotates the door 100 degrees around its hinge. Victory follows the sequence; retry resets it.

5.3 Texturing and animation appearance

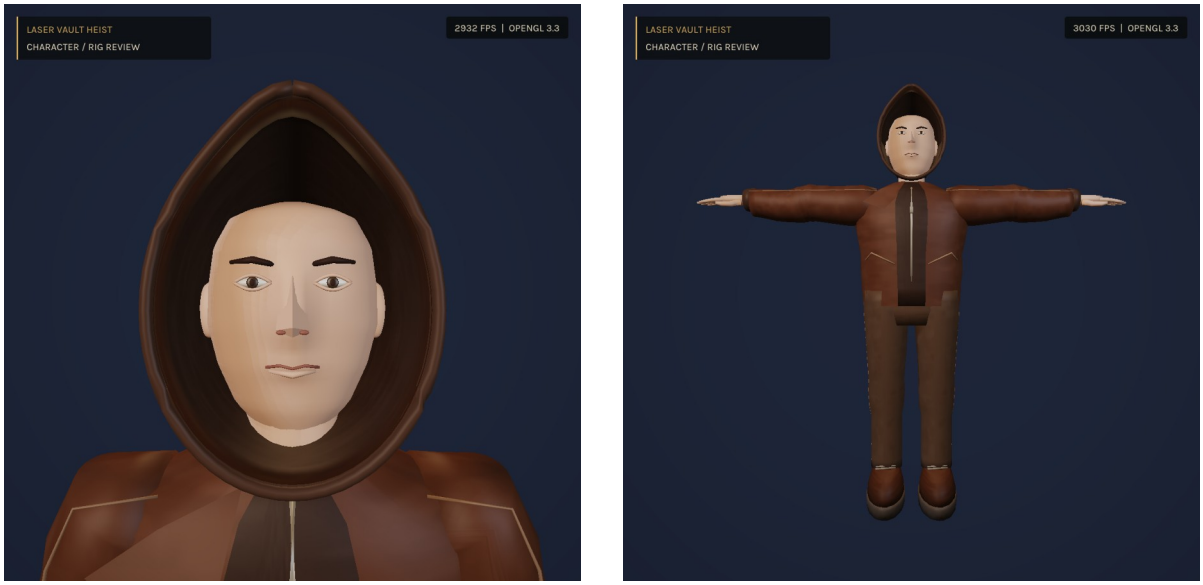


Figure 9. Readable face and differentiated brown clothing in neutral runtime views

Figure 9 verifies actual modeled skin, eyes, brows, nose, mouth and an open hood. The face is not a dark albedo mask. Jacket, sweatshirt, denim and shoes have distinct brown materials. Neutral inspection is brighter than the night scene so shape errors remain visible.

Seeded offline image generation creates original painted wood, plaster, stone, cloth and leather textures. Blender exports UVs and corner normals. The game uploads PNGs, generates mipmaps and uses trilinear minification with suitable wrapping. sRGB decoding precedes linear lighting. This addresses the sampling concern in [14] without adopting the synthesis algorithm of [15].

Pose checks sample all five clips, validate normalized weights and measure floor/height bounds. The lowest sampled point is 0.004 m above the floor. Visual review caught arms following the full torso lean in crouch; compensating the upper-arm rotation and bending elbows forward corrected it. Numerical validation and visual review cover different defects, within the authored motion range.

6 Testing and Evaluation

6.1 Testing methodology

Suite	Actual checks
gameplay_math	Boundary/rotated point-box queries; AABB false positive; clearance; interaction, pause, alarm, retry and victory
complete_gameplay_route	Lamp/CCTV actions; deliberate alarm/reset; legal six-beam route at six initial phases
demo_and_slow_frames	210 s Director route; three interactions; retained short input edges; 6 FPS versus 120 FPS collision

All three CTest suites passed in a clean Release build after unused vendor examples were removed. The executable ran from a different working directory and rendered the start menu and character review without GL errors. Actual frames were inspected at room, lamp, monitor, spotlight, alarm and victory milestones. GLB checks confirm isolated scene contents, a character skin and five named clips.

The demo records real wall-clock frames, including deliberate failure and a real retry. Its uninterrupted final attempt follows normal movement and collision rules. A frame ledger and gameplay-state CSV connect media with actual transitions. This is automated integration evidence, not a fabricated manual playtest.

Native UI automation could not obtain app approval, so human keyboard/mouse usability testing is not claimed. The TA hardware and exact course starter were not supplied. Live defense and author understanding require human assessment.

6.2 Performance evaluation

Hardware: Intel Core i5-13400F, NVIDIA GeForce RTX 5060, 32 GiB RAM, Windows x64 build 26200. OpenGL reports 3.3.0 NVIDIA 617.42. Three visible 1280 by 720 runs lasted 65 s each, excluding 5 s warm-up. VSync, frame limiting and video capture were off. Legal routes kept lasers, spotlights, CCTV, skinning, safe opening and retry active.

Run	Avg FPS	1% low	p50 ms	p95 ms	p99 ms
1	1281.7	753.0	0.747	1.005	1.199
2	1183.8	470.4	0.795	1.147	1.639
3	1215.8	686.3	0.790	1.070	1.290

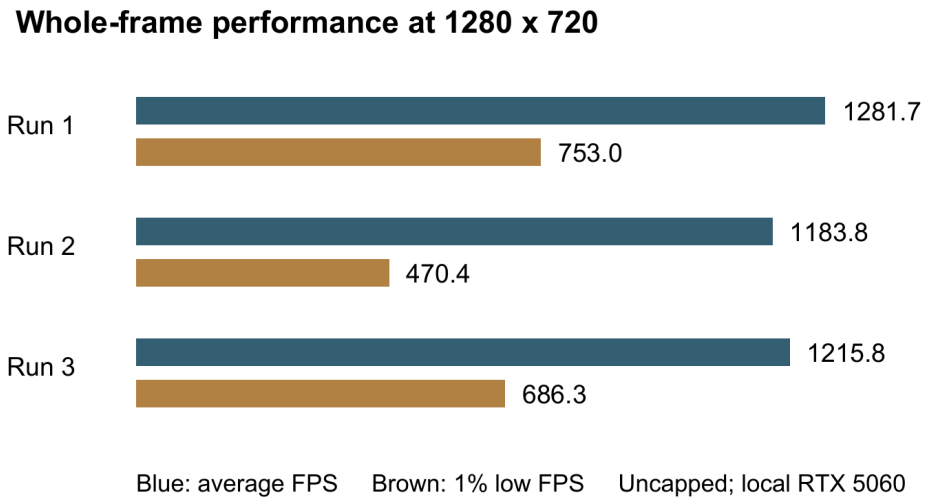


Figure 10. Average and low-tail performance from the three raw timing CSVs

Figure 10 uses average FPS = 1000 / mean(frame ms); 1% low = 1000 / mean(the slowest 1% of frame times). Percentiles use nearest rank. Whole-frame wall time includes simulation, draw submission, UI, logging and presentation; it is not isolated GPU time. Background desktop work was possible. Peak working sets were 109.7, 109.7 and 109.9 MiB; private memory peaks were 197.4, 197.9 and 192.0 MiB.

Local results exceed the 30 FPS mandatory threshold and 60 FPS rubric target. They cannot close the TA-machine wording. Raw data, sample counts and method remain in evidence/benchmarks; unknown hardware receives no inferred guarantee.

6.3 Issues and debugging

High render rates exposed two timing bugs: short interaction edges disappeared before a fixed update, and animation alternated between Walk and Idle when a render frame contained no simulation step. Retaining edges and persisting actual movement speed resolved them. Regression tests protect the edge case.

Early walk motion penetrated the floor. An analytic two-link leg solution now provides planted stance and lifted swing phases, with measured pose clearance. Neutral review also prompted the crouch-arm correction. Passing a weight-normalization check alone did not establish visual quality.

GLB export initially included multiple open scenes. Selection filtering plus active-scene export isolated the vault and thief. The enlarged CCTV initially sent linear color directly to ImGui and looked dark. A dedicated display conversion fixed it; the physical monitor retained the linear path.

An external desktop-capture experiment produced a white image instead of the OpenGL window and was discarded. The final recorder reads the real backbuffer into FFmpeg at wall-clock speed. Repeated sampling slots are logged explicitly; simulation is neither accelerated nor replaced with a manufactured success sequence.

Remaining limits are explicit: the small shadow map can alias, local lights have no shadow maps, the body proxy is approximate, LBS can lose joint volume outside checked poses, and recording is Windows-only. These do not become claims of physically exact shadows, continuous collision or verified universal platform support.

7 Conclusion and Future Work

Laser Vault Heist combines a complete game loop with inspectable graphics implementation. The executable contains modern OpenGL, custom shaders and buffers, hierarchical 3D objects, dynamic per-fragment lighting, textured UV geometry, keyboard/mouse input paths, three interactions and a visible FPS counter. All three topic stretch goals operate in one room.

Editable Blender sources, original rig/actions, verified exchange GLBs, native exports and scripts preserve the authoring chain. Collision, state and route tests passed. Continuous real-time video demonstrates failure, reset and safe-opening success. Three local performance runs show substantial headroom on the available RTX 5060 with a stated timing scope.

The main lesson is consistency between systems. The same laser matrix drives rendering and contact; crouch state drives camera, collider and pose; color-space decisions agree across CCTV and main display. Independent subsystem correctness does not automatically create a coherent game.

Future work begins with human input testing and the actual TA machine, then evidence-driven improvements: swept collision for faster motion, more stable shadow filtering, and stronger deformation for wider animation ranges. Accessibility controls or sound could be added if scope expands. Additional levels, combat and NPCs remain outside this submission.

No oral defense, unknown-machine test, classroom milestone or grade is asserted. The repository, report, deck, demonstration and defense guide provide concrete material for those remaining human checks.

8 References

[1] Bui Tuong Phong, "Illumination for computer generated pictures", Communications of the ACM, vol. 18, no. 6, pp. 311-317, 1975. DOI: 10.1145/360825.360839.
https://doi.org/10.1145/360825.360839

SJR 2025: Q1; Computer Science (miscellaneous).

[2] R. L. Cook, K. E. Torrance, "A Reflectance Model for Computer Graphics", ACM Transactions on Graphics, vol. 1, no. 1, pp. 7-24, 1982. DOI: 10.1145/357290.357293.
https://doi.org/10.1145/357290.357293

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[3] Christophe Schlick, "An Inexpensive BRDF Model for Physically-based Rendering", Computer Graphics Forum, vol. 13, no. 3, pp. 233-246, 1994. DOI: 10.1111/1467-8659.1330233.
https://doi.org/10.1111/1467-8659.1330233

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[4] Ladislav Kavan, Steven Collins, Jiří Žára, Carol O'Sullivan, "Geometric skinning with approximate dual quaternion blending", ACM Transactions on Graphics, vol. 27, no. 4, pp. 1-23, 2008. DOI: 10.1145/1409625.1409627.
https://doi.org/10.1145/1409625.1409627

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[5] L. Kavan, P.-P. Sloan, C. O'Sullivan, "Fast and Efficient Skinning of Animated Meshes", Computer Graphics Forum, vol. 29, no. 2, pp. 327-336, 2010. DOI: 10.1111/j.1467-8659.2009.01602.x.
https://doi.org/10.1111/j.1467-8659.2009.01602.x

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[6] Binh Huy Le, Zhigang Deng, "Smooth skinning decomposition with rigid bones", ACM Transactions on Graphics, vol. 31, no. 6, pp. 1-10, 2012. DOI: 10.1145/2366145.2366218.
https://doi.org/10.1145/2366145.2366218

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

8 References continued

[7] J.T. Klosowski, M. Held, J.S.B. Mitchell, H. Sowizral, K. Zikan, "Efficient collision detection using bounding volume hierarchies of k-DOPs", IEEE Transactions on Visualization and Computer Graphics, vol. 4, no. 1, pp. 21-36, 1998. DOI: 10.1109/2945.675649.
https://doi.org/10.1109/2945.675649

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[8] P. Jiménez, F. Thomas, C. Torras, "3D collision detection: a survey", Computers & Graphics, vol. 25, no. 2, pp. 269-285, 2001. DOI: 10.1016/S0097-8493(00)00130-8.
https://doi.org/10.1016/s0097-8493(00)00130-8

SJR 2025: Q2; Computer Graphics and Computer-Aided Design.

[9] D. Cohen-Or, Y.L. Chrysanthou, C.T. Silva, F. Durand, "A survey of visibility for walkthrough applications", IEEE Transactions on Visualization and Computer Graphics, vol. 9, no. 3, pp. 412-431, 2003. DOI: 10.1109/TVCG.2003.1207447.
https://doi.org/10.1109/tvcg.2003.1207447

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[10] Erik Reinhard, Michael Stark, Peter Shirley, James Ferwerda, "Photographic tone reproduction for digital images", ACM Transactions on Graphics, vol. 21, no. 3, pp. 267-276, 2002. DOI: 10.1145/566654.566575.
https://doi.org/10.1145/566654.566575

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[11] G.W. Larson, H. Rushmeier, C. Piatko, "A visibility matching tone reproduction operator for high dynamic range scenes", IEEE Transactions on Visualization and Computer Graphics, vol. 3, no. 4, pp. 291-306, 1997. DOI: 10.1109/2945.646233.
https://doi.org/10.1109/2945.646233

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[12] J. -M. Hasenfratz, M. Lapierre, N. Holzschuch, F. Sillion, "A Survey of Real -time Soft Shadows Algorithms", Computer Graphics Forum, vol. 22, no. 4, pp. 753-774, 2003. DOI: 10.1111/j.1467-8659.2003.00722.x.
https://doi.org/10.1111/j.1467-8659.2003.00722.x

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

8 References continued

[13] M. Teschner, S. Kimmerle, B. Heidelberger, G. Zachmann, L. Raghupathi, A. Fuhrmann, M.-P. Cani, F. Faure, N. Magnenat-Thalmann, W. Strasser, P. Volino, "Collision Detection for Deformable Objects", Computer Graphics Forum, vol. 24, no. 1, pp. 61-81, 2005. DOI: 10.1111/j.1467-8659.2005.00829.x. https://doi.org/10.1111/j.1467-8659.2005.00829.x

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[14] James F. Blinn, Martin E. Newell, "Texture and reflection in computer generated images", Communications of the ACM, vol. 19, no. 10, pp. 542-547, 1976. DOI: 10.1145/360349.360353. https://doi.org/10.1145/360349.360353

SJR 2025: Q1; Computer Science (miscellaneous).

[15] Sylvain Lefebvre, Hugues Hoppe, "Parallel controllable texture synthesis", ACM Transactions on Graphics, vol. 24, no. 3, pp. 777-786, 2005. DOI: 10.1145/1073204.1073261. https://doi.org/10.1145/1073204.1073261

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[16] Bruce Merry, Patrick Marais, James Gain, "Animation space: a truly linear framework for character animation", ACM Transactions on Graphics, vol. 25, no. 4, pp. 1400-1423, 2006. DOI: 10.1145/1183287.1183294. https://doi.org/10.1145/1183287.1183294

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[17] Alex Mohr, Michael Gleicher, "Building efficient, accurate character skins from examples", ACM Transactions on Graphics, vol. 22, no. 3, pp. 562-568, 2003. DOI: 10.1145/882262.882308. https://doi.org/10.1145/882262.882308

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

[18] Doug L. James, Christopher D. Twigg, "Skinning mesh animations", ACM Transactions on Graphics, vol. 24, no. 3, pp. 399-407, 2005. DOI: 10.1145/1073204.1073206. https://doi.org/10.1145/1073204.1073206

SJR 2025: Q1; Computer Graphics and Computer-Aided Design.

8.1 Technical sources and ranking provenance

[19] K. Narkowicz, "ACES Filmic Tone Mapping Curve", 2016. The fitted expression is offered under CC0 or MIT; this project uses CC0. https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/

Libraries: GLFW (glfw.org), GLAD (github.com/Dav1dde/glad), GLM (github.com/g-truc/glm), Dear ImGui (github.com/ocornut/imgui), stb (github.com/nothings/stb), Blender (blender.org), Karla by Jonathan Pinhorn (github.com/google/fonts/tree/main/ofl/karla). Versions and licenses are in dependency-lock.json, vendor files and LICENSES.

Project geometry, textures, rig and clips are original. Supplied concepts remain with the local brief, not redistributed as original assets. No purchased art, Mixamo clips or third-party audio was used. The brief supersedes the old AI-note requirement; ordinary attribution remains.

ACM Transactions on Graphics: SJR 2025 Q1. Ranking reproductions:

https://www.journalsbase.com/journals/acm-transactions-on-graphics

https://journalindexes.com/acm-transactions-on-graphics-07300301/

Computer Graphics Forum: SJR 2025 Q1. Ranking reproductions:

https://www.croris.hr/casopisi/casopis/10902

https://scienceaijournal.com/journals/computer-graphics-forum-0167-7055

IEEE Transactions on Visualization and Computer Graphics: SJR 2025 Q1. Ranking reproductions:

https://www.iit.comillas.edu/publicacion/info_revista/en/912/ieee-transactions-on-visualization-and-computer-graphics

https://researchcloud.net/journal/1941-0506

Computers & Graphics: SJR 2025 Q2. Ranking reproductions:

https://researchcloud.net/journal/0097-8493

https://akaturk.com/journals/25028?lang=en

Communications of the ACM: SJR 2025 Q1. Ranking reproductions:

https://www.iit.comillas.edu/publicacion/info_revista/en/16/Communications_of_the_ACM

https://scienceaijournal.com/journals/communications-of-the-acm-0001-0782

9 Appendix

9.1 Reproduction and evidence map

Evidence	Location
Build and controls	README.md; tools/build/build.ps1; CMakeLists.txt
Runtime mathematics	game.cpp; surface.vert; surface.frag; animation.cpp
Asset sources	art/blender; assets/exchange; tools/blender
Tests and performance	tests; evidence/benchmarks; character-pose-bounds.json
Requirement coverage	docs/requirements-evidence.md
Research audit	references-verified.json; source-audit.json
Demo and presentation	deliverables/video; deliverables/slides
Defense preparation	docs/defense-guide.md

9.2 Presentation and continuous demo

Ten slides include speaker notes for approximately 8.5 minutes. The 3:30 video follows: room/controls, lamp, CCTV, spotlights, alarm, reset, source diagnostics, continuous laser route and safe opening. Captions identify features and FPS stays visible. The final attempt has no cuts.

The defense guide covers MVP/normal matrices, hierarchy, inverse-box contact, Blinn-Phong, skinning, FBO feedback avoidance and fixed-step input. It distinguishes implementation from remaining native-input, TA-machine and course-starter checks. Actual submission and access invitations are separate human actions.